

AI-Minār

React Native Mobile App

Requirements Specification & Technical Reference

Version 1.0

Date 16 Feb 2026

Backend Django 6.0 REST API

Framework React Native / Expo

Table of Contents

1. Project Overview & Objectives
2. Brand Palette & Design Tokens
3. Typography & Iconography
4. Architecture & Tech Stack
5. API Reference & Endpoints
6. Authentication Flow (JWT)
7. Data Models (Frontend Interfaces)
8. Screen Inventory & Navigation Map
9. Screen-by-Screen Specifications
10. Component Library
11. Map Integration
12. Confidence System & Badges
13. State Management
14. Offline & Caching Strategy
15. Push Notifications
16. Testing Strategy
17. Accessibility & Internationalisation
18. Performance Targets
19. Deployment & CI/CD
20. Appendix — Backend Context
21. Authentication Screens (Login & Register)
22. Report Masjid Flow
23. Masjid Admin — Direct Submission & PDF Upload

24. **Community Verification (3-Signal Activation)**

01 Project Overview & Objectives

1.1 What is Al-Minār?

Al-Minār (المئارة — "The Lighthouse") is an open-source, community-driven global masjid registry. It helps Muslims discover nearby masjids, view prayer times, and verify each masjid's authenticity through a multi-layered **Confidence System** (C0 → C3).

1.2 Mobile App Goals

- **Discover** — Search, filter and browse masjids on an interactive map
- **Trust** — View confidence levels, verified badges, and signals
- **Pray** — Check prayer times (adhan & iqama) for any masjid
- **Contribute** — Send signals ("I prayed here", "Jummah observed") to upgrade confidence
- **Favourite** — Save and quickly access preferred masjids
- **Verify** — Scan/open badge verification links

1.3 Target Platforms

PLATFORM	MIN VERSION	NOTES
iOS	15.0+	iPhone & iPad
Android	API 24 (Android 7.0+)	Phones & Tablets

1.4 Existing Backend

The REST API is **already built and tested** (Django 6.0, DRF, 128 passing tests). The mobile app is a **consumer** of this API — no backend changes are required.

02 Brand Palette & Design Tokens

All colours are extracted from the Al-Minār brand logos. The palette is **gold & white** with charcoal for contrast. Use these exact hex values.

2.1 Primary Palette

Primary Dark (Charcoal)

#2C2C2C · rgb(44,44,44)

Primary (Dark Goldenrod)

#B8860B · rgb(184,134,11)

Primary Light (Goldenrod)

#DAA520 · rgb(218,165,32)

Accent Gold (Metallic Gold)

#D4AF37 · rgb(212,175,55)

Accent Light (Champagne)

#F5E6B8 ·
rgb(245,230,184)

Surface (Cream White)

#FDFBF7 ·
rgb(253,251,247)

2.2 Semantic & Utility Colours

White

#FFFFFF

Text Dark

#1A1A1A

Text Muted

#6B7280

Border

#E8E4DA

Success

#059669

Error / Danger

#DC2626

2.3 React Native Theme Object

```
// theme/colors.ts
export const colors = {
  primaryDark: '#2C2C2C',
  primary:      '#B8860B',
  primaryLight: '#DAA520',
  accent:       '#D4AF37',
  accentLight:  '#F5E6B8',
  surface:      '#FDFBF7',
  white:        'FFFFFF',
  text:         '#1A1A1A',
  textMuted:    '#6B7280',
  border:       '#E8E4DA',
  success:      '#059669',
  error:        '#DC2626',
  warning:      '#F59E0B',

  // Confidence badge colours
  c0Bg: '#F3F4F6', c0Text: '#6B7280',
  c1Bg: '#DBEAFE', c1Text: '#1E40AF',
  c2Bg: '#D1FAE5', c2Text: '#065F46',
  c3Bg: '#FEF3C7', c3Text: '#92400E', c3Border: '#F59E0B',
} as const;
```

2.4 Spacing & Radii

```
// theme/spacing.ts
export const spacing = {
  xs: 4, sm: 8, md: 16, lg: 24, xl: 32, xxl: 48,
} as const;

export const radii = {
  sm: 8, md: 12, lg: 20, xl: 28, full: 9999,
} as const;
```

2.5 Shadows

```
// theme/shadows.ts (iOS + Android)
export const shadows = {
  sm: {
    shadowColor: '#2C2C2C', shadowOffset: { width: 0, height: 1 },
    shadowOpacity: 0.08, shadowRadius: 3, elevation: 2,
  },
  md: {
    shadowColor: '#2C2C2C', shadowOffset: { width: 0, height: 4 },
```

```
        shadowOpacity: 0.10, shadowRadius: 16, elevation: 6,  
    },  
    lg: {  
        shadowColor: '#2C2C2C', shadowOffset: { width: 0, height: 12 },  
        shadowOpacity: 0.14, shadowRadius: 40, elevation: 12,  
    },  
} as const;
```

03 Typography & Iconography

3.1 Font Family

WEIGHT	USAGE	RN FONTWEIGHT
Inter Regular (400)	Body text, descriptions	'400'
Inter Medium (500)	Labels, nav items	'500'
Inter SemiBold (600)	Card titles, subtitles	'600'
Inter Bold (700)	Section headings, buttons	'700'
Inter ExtraBold (800)	Hero headings, stat numbers	'800'

Install via `expo-google-fonts` : `npx expo install @expo-google-fonts/inter expo-font`

3.2 Type Scale

```
// theme/typography.ts
export const fontSize = {
  xs: 11, // Badge labels, timestamps
  sm: 13, // Captions, filter chips
  md: 15, // Body text
  lg: 17, // Card titles
  xl: 22, // Section headings
  xxl: 28, // Screen titles
  hero: 34, // Hero / splash heading
} as const;
```

3.3 Icons

Use `@expo/vector-icons` (FontAwesome 5 set) — matches the web version exactly.

ICON	NAME	USAGE
	<code>mosque</code>	Masjid references
	<code>shield-alt</code>	C3 badge, security

	check-circle	C2 verified
	users	C1 community
	circle	C0 reported
	map-marker-alt	Location pins
	search	Search bars
	clock	Prayer times
	broadcast-tower	Signals
	certificate	Verified badges
	heart	Favourites
	compass	Explore tab

04 Architecture & Tech Stack

4.1 Recommended Stack

LAYER	TECHNOLOGY	VERSION	WHY
Framework	React Native + Expo	SDK 52+	Managed workflow, OTA updates
Language	TypeScript	5.x	Type safety, matches API interfaces
Navigation	Expo Router (file-based)	v4+	Tab + stack layouts, deep links
State	Zustand	5.x	Lightweight, no boilerplate
API Calls	TanStack Query (React Query)	5.x	Caching, pagination, offline
HTTP	Axios	1.x	Interceptors for JWT refresh
Maps	react-native-maps	1.x	Native MapView, markers, clusters
Secure Storage	expo-secure-store	—	JWT token storage
Forms	React Hook Form + Zod	—	Validation
Animations	react-native-reanimated	3.x	Smooth transitions
Testing	Jest + React Native Testing Library	—	Unit + component tests

4.2 Folder Structure

```
al-minar-mobile/  
├─ app/                                # Expo Router screens  
│   └─ (tabs)/                          # Bottom tab layout  
│       └─ _layout.tsx  
│       └─ index.tsx                    # Home
```

```

|   |   |─ explore.tsx           # Explore / Map
|   |   |─ favourites.tsx       # Favourites
|   |   └─ profile.tsx          # Profile / Settings
|   └─ masjid/
|       └─ [id].tsx              # Masjid detail
|   └─ verify/
|       └─ [token].tsx          # Badge verification
|   └─ auth/
|       └─ login.tsx
|       └─ register.tsx
|   └─ _layout.tsx              # Root layout
└─ components/
    └─ MasjidCard.tsx
    └─ ConfidenceBadge.tsx
    └─ VerifiedShield.tsx
    └─ PrayerTimeRow.tsx
    └─ SignalItem.tsx
    └─ SearchBar.tsx
    └─ FilterChip.tsx
    └─ MapPin.tsx
    └─ ...
└─ hooks/
    └─ useMasjids.ts
    └─ usePrayerTimes.ts
    └─ useSignals.ts
    └─ useBadges.ts
    └─ useAuth.ts
    └─ useFavourites.ts
└─ services/
    └─ api.ts                    # Axios instance + interceptors
    └─ masjidService.ts
    └─ authService.ts
    └─ ...
└─ store/
    └─ authStore.ts             # Zustand auth state
    └─ appStore.ts              # UI preferences
└─ theme/
    └─ colors.ts
    └─ typography.ts
    └─ spacing.ts
    └─ shadows.ts
└─ types/
    └─ api.ts                   # TypeScript interfaces
└─ utils/
    └─ formatters.ts
    └─ validators.ts
└─ assets/
    └─ images/
        └─ logo-1.png           # Full logo (gold on transparent)

```

	└─	logo-2.png	# Compact logo (navbar)
	└─	logo-3.png	# Alternate mark
	└─	logo-4.png	# Brand image
	└─	fonts/	

05 API Reference & Endpoints

Base URL: `https://your-api-domain.com/api/`

All responses use **PageNumberPagination** (25 per page) with `count` , `next` , `previous` , `results` .

5.1 Authentication

METHOD	ENDPOINT	BODY	RETURNS
POST	<code>/api/token/</code>	<code>{ username, password }</code>	<code>{ access, refresh }</code>
POST	<code>/api/token/refresh/</code>	<code>{ refresh }</code>	<code>{ access, refresh }</code> (rotated)

Token Lifetimes: Access = 30 min, Refresh = 7 days. Refresh tokens rotate and blacklist old ones automatically.

5.2 Masjids

METHOD	ENDPOINT	AUTH	NOTES
GET	<code>/api/masjids/</code>	Public	List all · <code>?search= ?</code> <code>isActive= ?</code> <code>ordering=</code>
GET	<code>/api/masjids/{uuid}/</code>	Public	Detail (includes nested <code>confidenceRecord</code> & <code>locationRecord</code>)
POST	<code>/api/masjids/</code>	JWT	Create new masjid
PUT/PATCH	<code>/api/masjids/{uuid}/</code>	JWT	Update
DELETE	<code>/api/masjids/{uuid}/</code>	JWT	Delete
GET	<code>/api/masjids/{uuid}/prayer_times/</code>	Public	Prayer time records for masjid

GET	/api/masjids/{uuid}/signals/	Public	Signals for masjid
GET	/api/masjids/{uuid}/badges/	Public	Verified badges for masjid

5.3 Location Records

METHOD	ENDPOINT	AUTH	NOTES
GET	/api/location-records/	Public	?country= ?city= ?masjid= ?search=
POST	/api/location-records/	JWT	Create

5.4 Confidence Records

METHOD	ENDPOINT	AUTH	NOTES
GET	/api/confidence-records/	Public	?confidenceLevel= ?masjid=
POST	/api/confidence-records/	Admin	Admin only

5.5 Signals

METHOD	ENDPOINT	AUTH	NOTES
GET	/api/signals/	Public	?masjid= ?signalType= ?sourceType=
POST	/api/signals/	JWT	Creates signal → auto-runs <code>process_signal()</code> to upgrade confidence

Signal Types: PRAYED , JUMMAH , ACTIVE , ADMIN_VERIFY

Source Types: USER , ADMIN , SYSTEM

5.6 Prayer Times

METHOD	ENDPOINT	AUTH	NOTES
--------	----------	------	-------

GET	/api/prayer-time-records/	Public	?masjid= ?date= ?modelType=
GET	/api/prayers/	Public	Lookup: fajr, dhuhr, asr, maghrib, isha
GET	/api/prayer-times/	Public	Individual prayer time entries

5.7 Badges & Verification

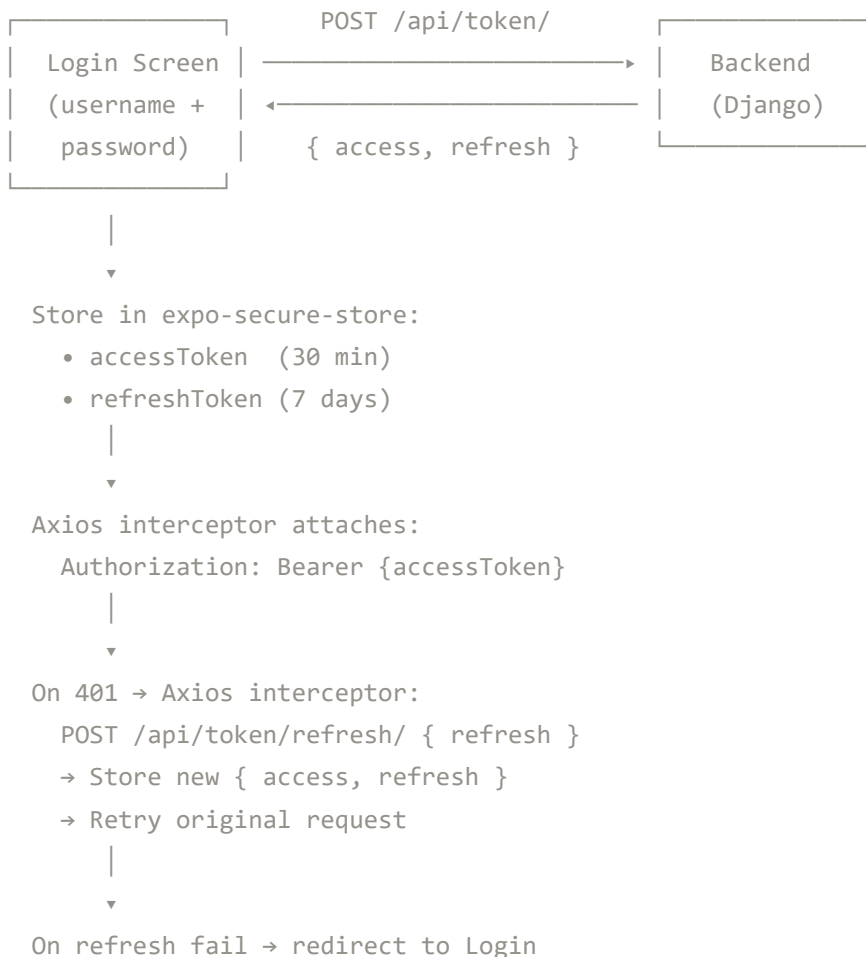
METHOD	ENDPOINT	AUTH	NOTES
GET	/api/badges/	Public	?masjid= ?isActive= ?isRevoked=
POST	/api/badges/{uuid}/revoke/	Admin	Revoke a badge
GET	/api/verify/{uuid}/	Public	Returns { valid, masjid, masjidID, issuedAt, expiresAt }

5.8 Favourites & Users

METHOD	ENDPOINT	AUTH	NOTES
GET	/api/favourites/	JWT	?user= ?masjid=
POST	/api/favourites/	JWT	{ user, masjid }
DELETE	/api/favourites/{uuid}/	JWT	Un-favourite
GET	/api/users/	JWT	Regular users

06 Authentication Flow (JWT)

6.1 Flow Diagram



6.2 Key Implementation Notes

- Use `expo-secure-store` — NOT `AsyncStorage` — for token storage (encrypted at rest)
- Refresh tokens rotate: the old refresh token is **blacklisted** after use
- Queue concurrent 401 retries so only one refresh request fires at a time
- The app should support an **unauthenticated guest mode** — browse masjids, view prayer times — but require login to send signals, manage favourites, etc.

07 Data Models (TypeScript Interfaces)

```
// types/api.ts
```

```
export interface Masjid {  
  masjidID:      string;    // UUID  
  name:          string;  
  description:   string | null;  
  isActive:      boolean;  
  confidenceRecord: ConfidenceRecord | null;  
  locationRecord: LocationRecord | null;  
  created_at:    string;    // ISO datetime  
  updated_at:    string;  
}
```

```
export interface ConfidenceRecord {  
  crID:          string;  
  confidenceLevel: 0 | 1 | 2 | 3;  
  masjid:        string;  
  lastConfirmationDate: string;  
  decayDate:     string | null;  
  created_at:    string;  
  updated_at:    string;  
}
```

```
export interface LocationRecord {  
  lrID:      string;  
  latitude:  number;  
  longitude: number;  
  masjid:    string;  
  city:     string | null;  
  country:  string;  
  region:   string | null;  
  created_at: string;  
  updated_at: string;  
}
```

```
export interface PrayerTimeRecord {  
  ptrID:      string;  
  masjid:     string;  
  modelType:  'FULL_TIMETABLE' | 'IQAMA_ONLY' | 'ADHAN_ONLY' | 'UNKNOWN';  
  isVariable: boolean;  
  date:       string;    // YYYY-MM-DD  
  lastUpdated: string;  
  prayers:    PrayerTime[];
```

```
    created_at: string;
    updated_at: string;
}

export interface PrayerTime {
    prayerTimeID: string;
    record: string;
    prayer: Prayer;
    adhan_time: string | null; // HH:MM:SS
    iqama_time: string | null;
    created_at: string;
    updated_at: string;
}

export interface Prayer {
    id: number;
    name: 'fajr' | 'dhuhr' | 'asr' | 'maghrib' | 'isha';
    created_at: string;
    updated_at: string;
}

export interface Signal {
    signalID: string;
    masjid: string;
    user: string | null;
    signalType: 'PRAYED' | 'JUMMAH' | 'ACTIVE' | 'ADMIN_VERIFY';
    sourceType: 'USER' | 'ADMIN' | 'SYSTEM';
    description: string | null;
    created_at: string;
    updated_at: string;
}

export interface VerifiedBadge {
    badgeID: string;
    token: string;
    masjid: string;
    issuedBy: string;
    issueDate: string;
    expiryDate: string | null;
    isActive: boolean;
    isRevoked: boolean;
    lastCheckedAt: string | null;
    created_at: string;
    updated_at: string;
}

export interface FavouriteMasjid {
    favID: string;
    user: string;
```

```
    masjid:    string;
    created_at: string;
    updated_at: string;
}
```

```
export interface VerifyResponse {
    valid:    boolean;
    masjid:   string;
    masjidID: string;
    issuedAt: string;
    expiresAt: string | null;
}
```

```
export interface PaginatedResponse<T> {
    count:    number;
    next:     string | null;
    previous: string | null;
    results:  T[];
}
```

08 Screen Inventory & Navigation Map

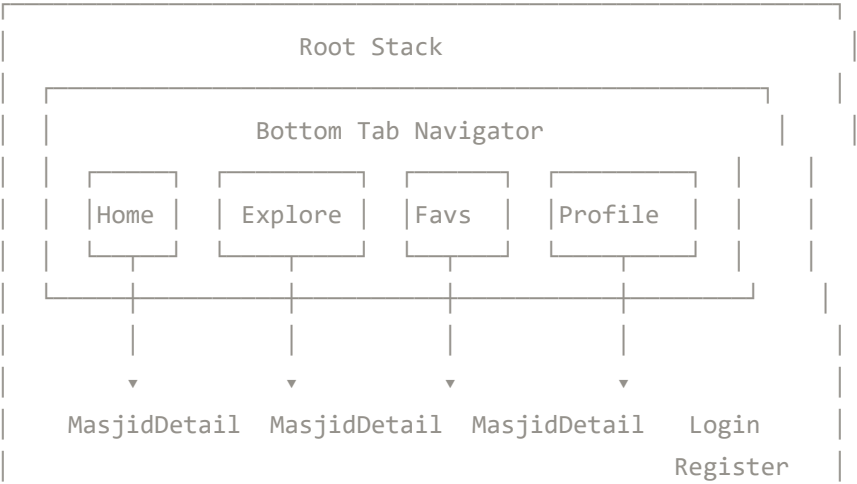
8.1 Bottom Tab Bar

TAB	ICON	SCREEN	ROUTE
Home	home	HomeScreen	/(tabs)/
Explore	compass	ExploreScreen	/(tabs)/explore
Favourites	heart	FavouritesScreen	/(tabs)/favourites
Profile	user	ProfileScreen	/(tabs)/profile

8.2 Stack Screens (pushed from tabs)

SCREEN	ROUTE	TRIGGER
MasjidDetailScreen	/masjid/[id]	Tap masjid card or map pin
BadgeVerifyScreen	/verify/[token]	Deep link or scan QR
LoginScreen	/auth/login	Tap "Login" or protected action
RegisterScreen	/auth/register	Tap "Register"

8.3 Navigation Flow Diagram



BadgeVerify (deep link from any context)

09 Screen-by-Screen Specifications

9.1 — HomeScreen TAB 1

Purpose: Landing screen — hero branding, quick search, stats, recently added masjids.

API calls:

- `GET /api/masjids/?ordering=-created_at&page_size=6` — featured masjids

Sections (top → bottom):

1. **Hero area** — Gradient background (`charcoal`), logo (`logo-1.png`), tagline "Discover & Trust Your Local Masjid", gold accent text
2. **Search bar** — Tapping navigates to Explore tab with keyboard open
3. **Stat strip** — 3 stat cards: Masjids count, Countries count, Verified count (computed from API or cached)
4. **Confidence explainer** — 4 horizontal cards (C0–C3) showing badge colours & descriptions
5. **Recently Added** — Horizontal `FlatList` of `MasjidCard` components
6. **CTA** — "Explore Masjids" button → navigate to Explore tab

9.2 — ExploreScreen TAB 2

Purpose: Search, filter, and discover masjids on a map and list.

API calls:

- `GET /api/masjids/?search={q}&ordering=-created_at`
- `GET /api/location-records/?country={code}`

Layout:

1. **Search bar** — sticky at top, debounced (300ms)
2. **Filter chips row** — Horizontal `ScrollView`
 - Confidence: C0, C1, C2, C3 (toggle)
 - Region: All, UK, USA, Saudi Arabia, Egypt, Pakistan, India, Malaysia, Turkey
3. **Map / List toggle** — Segmented control to switch between:

- **Map view** — `react-native-maps` MapView with gold pins, popup on tap → navigate to detail
- **List view** — Vertical `FlatList` of `MasjidCard`

4. **Result count** — "12 masjids found"

9.3 — MasjidDetailScreen [STACK](#)

Purpose: Full detail view for a single masjid.

API calls:

- `GET /api/masjids/{uuid}/`
- `GET /api/masjids/{uuid}/prayer_times/`
- `GET /api/masjids/{uuid}/signals/`
- `GET /api/masjids/{uuid}/badges/`

Sections (ScrollView):

1. **Header** — Gradient bar, masjid name, location, confidence badge, active status, verified shield (if C2+)
2. **About card** — Description text
3. **Prayer Times card** — 5 rows (Fajr–Isha), each showing adhan & iqama chips
4. **Location card** — Small embedded map (static or interactive), lat/lng display
5. **Signals card** — Last 10 signals, each with icon, type, source, date
6. **Badges card** — List of badges, each showing status (Valid/Revoked), issue/expiry dates
7. **Confidence card** — Large "C2" display, level name, decay date
8. **Metadata card** — Masjid ID (truncated), created/updated dates
9. **Action bar** (bottom) — "Send Signal" button (authenticated), "Favourite" heart toggle, "Share" button

9.4 — FavouritesScreen [TAB 3](#)

Purpose: Quick access to saved masjids.

API calls:

- `GET /api/favourites/?user={username}`
- Then fetch masjid details for each favourite

Behaviour:

- Requires authentication — show login prompt if guest
- Vertical list of **MasjidCard** with swipe-to-delete (un-favourite)
- Empty state: mosque icon + "No favourites yet — explore masjids to save them"

9.5 — ProfileScreen TAB 4

Purpose: User account & app settings.

Sections:

1. **User info** — Username, email (from stored auth)
 2. **My Signals** — Count of signals sent
 3. **Settings** — Dark mode toggle, notification preferences, default map region
 4. **About Al-Minār** — Version, link to GitHub, "The Lighthouse" mission text
 5. **Logout button**
- If not logged in: show Login / Register buttons instead

9.6 — LoginScreen AUTH

Purpose: JWT login.

- Logo at top, gold gradient "Login" button
- Fields: username, password
- Validation: required, min 3 chars
- API: **POST** /api/token/
- On success: store tokens → navigate to previous screen or Home
- Link to Register screen

9.7 — RegisterScreen AUTH

Purpose: Create user account.

- Fields: username, email, password, confirm password
- Validation: email format, password match, min 8 chars

- API: `POST /api/users/` → then auto-login via `/api/token/`

9.8 — BadgeVerifyScreen [STACK](#) / [DEEP LINK](#)

Purpose: Verify a masjid badge by token (e.g., scanned QR code).

- API: `GET /api/verify/{token}/`
- Loading state: spinner + "Verifying Badge..."
- Valid: Green check icon, masjid name, issue/expiry dates, "View Masjid" button
- Invalid: Red X icon, "Badge Invalid" message
- Not found: Question mark icon, "Badge Not Found"
- Deep link: `alminar://verify/{token}` or `https://alminar.org/verify/{token}/page/`

10 Component Library

COMPONENT	PROPS	DESCRIPTION
MasjidCard	<code>masjid: Masjid ,</code> <code>onPress: () => void</code>	Card with name, location, confidence badge, active status, "View Details" link. White bg, <code>radii.lg</code> , gold top band (6px gradient), shadow.sm on hover/press.
ConfidenceBadge	<code>level: 0 1 2 3 , size?:</code> <code>'sm' 'md'</code>	Pill badge with icon + label. Colours: C0=gray, C1=blue, C2=green, C3=gold+border.
VerifiedShield	<code>variant?:</code> <code>'gold' 'default' ,</code> <code>size?: number</code>	Shield-shaped badge (rounded bottom). Gold gradient for verified, charcoal for default. Certificate icon + "Verified" label.
PrayerTimeRow	<code>prayer: PrayerTime</code>	Row with prayer name + icon, adhan chip (champagne bg), iqama chip (green bg).
SignalItem	<code>signal: Signal</code>	Row with broadcast icon in rounded square, signal type, source type, date.
SearchBar	<code>value , onChangeText ,</code> <code>placeholder</code>	Rounded white bar with search icon, gold search button.
FilterChip	<code>label , active ,</code> <code>onPress</code>	Pill toggle. Active = charcoal bg + white text. Inactive = white bg + border.
MapPin	<code>level: 0 1 2 3</code>	Custom marker for react-native-maps. Gold circle (12px) with white border + shadow.
StatCard	<code>number , label</code>	Large gold number + uppercase muted label below.
InfoCard	<code>title , icon ,</code> <code>children</code>	White card with icon + uppercase section title (muted), content area. Used on detail screen.

GoldButton

title , onPress ,
icon? , variant?:
'solid'|'outline'

Solid: gold gradient bg, white text.
Outline: transparent bg, white border.
Full-width pill shape.

EmptyState

icon , title , message

Centred icon (large, muted), title,
description. Used for empty lists.

11 Map Integration

11.1 Library

Use `react-native-maps` with **Apple Maps** (iOS) and **Google Maps** (Android).

11.2 Features

FEATURE	IMPLEMENTATION
Custom markers	Gold circle pins (12px) using custom <code>MapPin</code> component
Marker clustering	<code>react-native-map-clustering</code> for performance at scale
Callout popup	Masjid name, city, confidence badge, "View Details" link
User location	<code>expo-location</code> for "Near Me" functionality
Fit bounds	Auto-zoom to fit all visible markers with 40px padding
Detail mini-map	Static <code>MapView</code> (scrollEnabled=false) on masjid detail page

11.3 Initial Region

```
const initialRegion = {  
  latitude: 25,  
  longitude: 45,  
  latitudeDelta: 80,  
  longitudeDelta: 80,  
};
```

12 Confidence System & Badges

12.1 Confidence Levels

LEVEL	NAME	TRIGGER	DECAY	BADGE COLOUR
C0	Community Reported	Initial entry	Does not decay further	Gray: #F3F4F6 / #6B7280
C1	Community Confirmed	3+ unique user signals in 30 days	365 days → C0	Blue: #DBEAFE / #1E40AF
C2	Masjid Verified	Admin signal	180 days → C1	Green: #D1FAE5 / #065F46
C3	Actively Maintained	Admin signal when already C2	90 days → C2	Gold: #FEF3C7 / #92400E / border #F59E0B

12.2 Signal Sending (Mobile)

On the MasjidDetailScreen, authenticated users see a **"Send Signal"** button. Tapping opens a bottom sheet with:

- 🧎 I prayed here — signalType: PRAYED
- 🕌 Jummah observed — signalType: JUMMAH
- ✅ Masjid is active — signalType: ACTIVE

Each sends `POST /api/signals/` with `sourceType: 'USER'`. Backend automatically runs `process_signal()`.

12.3 Verified Badges

Masjids at C2+ may have `VerifiedBadge` records. Display the gold `VerifiedShield` component on their detail page. The badge `token` UUID can be encoded as a QR code for physical display.

13 State Management

13.1 Zustand Stores

STORE	STATE	PERSISTED?
<code>authStore</code>	<code>accessToken</code> , <code>refreshToken</code> , <code>user</code> , <code>isAuthenticated</code> , <code>login()</code> , <code>logout()</code> , <code>refreshTokens()</code>	Yes (expo-secure-store for tokens)
<code>appStore</code>	<code>isDarkMode</code> , <code>defaultRegion</code> , <code>notificationsEnabled</code>	Yes (AsyncStorage)

13.2 TanStack Query Keys

```
// hooks/queryKeys.ts
export const queryKeys = {
  masjids: (params?: object) => ['masjids', params],
  masjid: (id: string) => ['masjid', id],
  prayerTimes: (id: string) => ['prayerTimes', id],
  signals: (id: string) => ['signals', id],
  badges: (id: string) => ['badges', id],
  favourites: (user: string) => ['favourites', user],
  verify: (token: string) => ['verify', token],
  locations: (params?: object) => ['locations', params],
} as const;
```

14 Offline & Caching Strategy

DATA	CACHE DURATION	STRATEGY
Masjid list	5 min <code>staleTime</code>	TanStack Query cache, refetch on focus
Masjid detail	10 min	Cache per UUID, background refetch
Prayer times	30 min	Longer cache — data changes infrequently
Favourites	Always fresh	Invalidate on add/remove, optimistic updates
Search results	2 min	Short cache, new search replaces
Offline mode	—	TanStack Query persister (<code>@tanstack/query-async-storage-persister</code>) for offline read access to previously loaded data

15 Push Notifications (Future)

Not required for MVP, but plan for:

- **Prayer time reminders** — for favourited masjids
- **Confidence change alerts** — "Your masjid upgraded to C2!"
- **Badge expiry warnings** — 7 days before a badge expires
- **Decay warnings** — "This masjid's confidence will decay in 14 days"

Use `expo-notifications` + backend would need a notification service (e.g., Firebase Cloud Messaging).

16 Testing Strategy

LAYER	TOOL	TARGET
Unit	Jest	Utility functions, formatters, validators, store logic
Component	React Native Testing Library	All components in §10 — render, props, snapshots
Integration	MSW (Mock Service Worker)	API hooks with mocked responses
E2E	Detox or Maestro	Full user flows: search → detail → send signal → favourite

Minimum Test Coverage Targets

- Components: 80%+
- Hooks/services: 90%+
- Stores: 95%+
- E2E: Core happy paths (login, search, detail, signal, favourite, verify)

17 Accessibility & Internationalisation

17.1 Accessibility (a11y)

- All interactive elements must have `accessibilityLabel` and `accessibilityRole`
- Confidence badges: announce level name (e.g., "Community Confirmed, Confidence Level 1")
- Map pins: `accessibilityLabel="{masjid name}, {city}"`
- Minimum touch target: 44×44pt
- Colour contrast: all text meets WCAG 2.1 AA (4.5:1 ratio)
- Support Dynamic Type (iOS) and font scaling (Android)

17.2 Internationalisation (i18n)

- Use `expo-localization` + `i18next`
- Initial languages: **English, Arabic** (RTL support via `I18nManager.forceRTL()`)
- Arabic masjid names should be rendered with proper RTL alignment
- Prayer names (Fajr, Dhuhr, etc.) should have Arabic translations

18 Performance Targets

METRIC	TARGET	HOW
App launch → interactive	< 2s	Splash screen, lazy load tabs, pre-warm auth
Search response render	< 500ms	Debounced input, skeleton loaders
Map with 100+ markers	60fps scroll	Marker clustering, limit to visible region
FlatList scroll	No jank	<code>getItemLayout</code> , <code>windowSize</code> , <code>maxToRenderPerBatch</code>
Bundle size	< 15 MB (iOS), < 12 MB (Android)	Tree shaking, lazy imports, optimize images
Image assets	WebP format	Convert brand PNGs to WebP for mobile

19 Deployment & CI/CD

19.1 Build & Submit

STEP	TOOL	COMMAND
Dev build (iOS)	Expo	<code>npx expo run:ios</code>
Dev build (Android)	Expo	<code>npx expo run:android</code>
Production build	EAS Build	<code>eas build --platform all</code>
OTA update	EAS Update	<code>eas update --branch production</code>
Submit to stores	EAS Submit	<code>eas submit --platform all</code>

19.2 Environment Variables

```
// .env
API_BASE_URL=https://api.alminar.org
GOOGLE_MAPS_API_KEY=your-key-here
```

19.3 CI Pipeline (GitHub Actions)

1. Lint (`eslint`) + Type check (`tsc --noEmit`)
2. Unit & component tests (`jest`)
3. Build check (`eas build --platform all --non-interactive`)
4. Deploy preview (EAS Update to preview branch)
5. Production release on tag push

20 Appendix — Backend Context

20.1 Backend Stack

COMPONENT	DETAIL
Framework	Django 6.0.2, Python 3.13
API	Django REST Framework
Auth	SimpleJWT (30min access, 7day refresh, rotate+blacklist)
Filtering	django-filter + SearchFilter + OrderingFilter
CORS	django-cors-headers (open in dev, whitelist in prod)
Database	SQLite (dev) → PostgreSQL (prod)
Tests	128 tests, all passing
Admin	django-jazzmin themed admin panel

20.2 Backend Models (12)

MODEL	PK	KEY FIELDS
AdminUser	username	email
RegularUser	username	email
Masjid	masjidID (UUID)	name, description, isActive, FK→ConfidenceRecord, FK→LocationRecord
ConfidenceRecord	crID (UUID)	confidenceLevel (0-3), FK→Masjid, lastConfirmationDate, decayDate
LocationRecord	lrID (UUID)	latitude, longitude, city, country, region, FK→Masjid
PrayerTimeRecord	ptrID (UUID)	FK→Masjid, modelType, isVariable, date, unique(masjid,date)

Prayer	id (auto)	name (fajr dhuhr asr maghrib isha), unique
PrayerTime	prayerTimeID (UUID)	FK→PrayerTimeRecord, FK→Prayer, adhan_time, iqama_time
Signal	signalID (UUID)	FK→Masjid, FK→RegularUser, signalType, sourceType
VerifiedBadge	badgeID (UUID)	token (UUID unique), FK→Masjid, FK→AdminUser, isActive, isRevoked, expiryDate
MasjidAdmin	masjidAdminID (UUID)	FK→RegularUser, FK→Masjid, verifiedIdentity, unique(user,masjid)
FavouriteMasjid	favID (UUID)	FK→RegularUser, FK→Masjid, unique(user,masjid)

20.3 Business Logic (Services Layer)

FUNCTION	TRIGGER	BEHAVIOUR
<code>process_signal(signal)</code>	Signal creation	ADMIN signal → C2 (or C3 if already C2). USER signals → C1 when ≥3 unique users in 30 days.
<code>decay_confidence(record)</code>	Scheduled job	If past decayDate, drop level by 1. C3→90d, C2→180d, C1→365d, C0→never.
<code>check_badge_validity(badge)</code>	Badge verification	Invalid if: revoked, inactive, expired, masjid C<2, masjid inactive. Auto-deactivates.

20.4 Brand Assets

FILE	SIZE	DESCRIPTION	USAGE
<code>1.png</code>	14 KB	Full logo (gold on transparent)	Hero, splash screen, about page
<code>2.png</code>	13 KB	Compact logo	Navbar, tab bar header, verify page
<code>3.png</code>	126 KB	Alternate mark	CTA sections, loading screens
<code>4.png</code>	64 KB	Brand image	About/mission page

Note: Convert all PNGs to WebP and provide @1x, @2x, @3x variants for React Native.
Place in `assets/images/` .

21 Authentication Screens (Login & Register)

21.1 Login Screen

LoginScreen

PROPERTY	VALUE
Route	<code>/(auth)/login</code>
Auth Required	NO
API Endpoint	<code>POST /api/token/</code>

UI Elements

- Brand logo (compact — `2.png`)
- "Welcome Back" heading + subtext
- Username input with user icon
- Password input with lock icon (show/hide toggle)
- "Remember Me" toggle
- Primary CTA — "Sign In" (gold gradient button)
- Divider — "New to Al-Minār?"
- Secondary CTA — "Create an Account" (outline button → Register screen)
- Error alert component (red background) for invalid credentials

Logic

1. `POST /api/token/` with `{username, password}`
2. On success → store `access` & `refresh` tokens in `SecureStore`
3. Fetch user profile: `GET /api/users/{username}/`
4. Check masjid-admin status: `GET /api/masjid-admins/?user={username}`
5. Store user object in Zustand `authStore`
6. Navigate to Home tab

Token Refresh

Use an Axios interceptor / TanStack Query `onError` handler. On 401 → `POST /api/token/refresh/` with the stored refresh token. If refresh fails → clear store,

redirect to Login.

21.2 Register Screen

RegisterScreen

PROPERTY	VALUE
Route	<code>/(auth)/register</code>
Auth Required	NO
API Endpoint	<code>POST /api/users/</code> + auto-login via <code>POST /api/token/</code>

UI Elements

- Brand logo
- "Join Al-Minār" heading
- **Account type selector** — two tappable cards:
 - **REGULAR USER** — "Report & verify masjids"
 - **MASJID ADMIN** — "Manage your masjid directly"
- Username input (min 3 chars)
- Email input
- Password input (min 8 chars)
- Confirm password input
- Community guidelines checkbox
- Info banner (if Masjid Admin selected): "You can add your masjid directly after registration"
- Primary CTA — "Create Account"
- Link to Login screen

Logic

1. Validate passwords match, min lengths met
2. `POST /api/users/` with `{username, email, password}`
3. On success → auto-login via `POST /api/token/`
4. Store tokens + user info (including `accountType`)
5. If `masjid_admin` → navigate to Report Masjid screen

6. If `user` → navigate to Home

21.3 Auth State Interface

```
// stores/authStore.ts
interface AuthUser {
  username: string;
  email: string;
  accountType: 'user' | 'masjid_admin';
}

interface AuthState {
  accessToken: string | null;
  refreshToken: string | null;
  user: AuthUser | null;
  isLoggedIn: boolean;
  login: (username: string, password: string) => Promise<void>;
  register: (data: RegisterPayload) => Promise<void>;
  logout: () => void;
  refreshAccessToken: () => Promise<boolean>;
}
```

22 Report Masjid Flow

22.1 Overview

Any logged-in user can report a masjid they know about. The flow differs based on user role:

ROLE	BEHAVIOUR	VISIBILITY
REGULAR USER	Creates masjid with <code>isActive: false</code> . Submits an initial <code>ACTIVE</code> signal.	Appears on site only after 3 unique community signals within 30 days.
MASJID ADMIN	Creates masjid with <code>isActive: true</code> . Submits an <code>ADMIN_VERIFY</code> signal. Uploads PDF letterhead.	Immediately visible. Confidence starts at C2.

22.2 Report Screen Specification

ReportMasjidScreen

PROPERTY	VALUE
Route	<code>/(tabs)/report</code>
Auth Required	YES
API Endpoints	<code>POST /api/masjids/</code> <code>POST /api/location-records/</code> <code>POST /api/signals/</code> <code>POST /api/masjid-admins/</code> (admin only)

UI Sections

- Auth Wall** — If not logged in, show lock icon + CTA to login/register
- Role Banner** — Blue for regular user (explains 3-signal rule), green for admin (explains direct add)
- Masjid Details** — Name (required), description (optional)

4. **Location** — City (required), country dropdown (required), region (optional), latitude/longitude (required)
5. **Map Picker** — Interactive map. Tap to set coordinates. Marker moves on tap.
Uses `react-native-maps`
6. **PDF Upload** (admin only) — Drag & drop zone / file picker. Accept `.pdf` only, max 5 MB
7. **Submit Button** — Gold gradient, full width

22.3 Submission API Sequence

// Regular User submission

1. POST `/api/masjids/` → { name, description, isActive: false }
2. POST `/api/location-records/` → { masjid: id, latitude, longitude, city, country, region }
3. POST `/api/signals/` → { masjid: id, signalType: "ACTIVE", sourceType: "USER", }

// Masjid Admin submission

1. POST `/api/masjids/` → { name, description, isActive: true }
2. POST `/api/location-records/` → { masjid: id, latitude, longitude, city, country, region }
3. POST `/api/signals/` → { masjid: id, signalType: "ADMIN_VERIFY", sourceType: 'ADMIN' }
4. POST `/api/masjid-admins/` → { user: username, masjid: id, verifiedIdentity: false }
5. Upload PDF letterhead (see §23)



23 Masjid Admin — Direct Submission & PDF Upload

23.1 Admin Flow

When a user registers as a **Masjid Admin**, they gain the ability to:

- Add a masjid directly (immediately visible, `isActive: true`)
- Upload a PDF of the masjid's official letterhead for identity verification
- The backend processes the `ADMIN_VERIFY` signal, which bumps confidence to **C2**
- If the masjid already has recent admin activity, confidence may reach **C3**

23.2 PDF Upload Component

PDFUploadWidget

PROPERTY	SPEC
Accepted type	<code>application/pdf</code>
Max file size	5 MB
UI	Dashed border drop zone, cloud-upload icon, file name badge on selection
Mobile	Use <code>expo-document-picker</code> with <code>type: "application/pdf"</code>

Backend suggestion: The PDF upload currently requires a new backend endpoint. See §24 "Community Verification" for the complete list of suggested backend changes. On the frontend, store the file reference and submit via `multipart/form-data` when the backend is ready.

23.3 Verification Status

After submission, the Masjid Admin entry is created with `verifiedIdentity: false` . The platform team reviews the PDF letterhead and sets `verifiedIdentity: true` via the admin

panel. The mobile app should show a status indicator:

STATUS	DISPLAY	BADGE
verifiedIdentity: false	"Verification Pending"	PENDING
verifiedIdentity: true	"Identity Verified"	VERIFIED

24 Community Verification (3-Signal Activation)

24.1 How It Works

When a **regular user** reports a masjid, it is created with `isActive: false` and does not appear in the main explore feed. To become visible:

1. The reporting user's initial signal counts as signal #1
2. Two more **unique users** must verify the masjid within 30 days
3. Each verification creates a `Signal` with `signalType: "ACTIVE"` and `sourceType: "USER"`
4. The backend's `process_signal()` counts unique users — when ≥ 3 , the masjid's `ConfidenceRecord` is upgraded to **C1**
5. A Masjid Admin sending an `ADMIN_VERIFY` signal at any point immediately makes the masjid visible (C2)

24.2 Pending Masjids Screen

PendingMasjidsView

Shown on the Report screen below the submission form (for regular users only).

PROPERTY	VALUE
Data Source	<code>GET /api/masjids/?isActive=false&ordering=-created_at</code>
Card Layout	Compact card: name, location, description, "I Can Verify This Masjid" button
Verify Action	<code>POST /api/signals/ → {masjid, signalType: "ACTIVE", sourceType: "USER", user}</code>

UI Behaviour

- Button shows loading spinner during API call
- On success → button changes to green "Verified — Thank You!" (disabled)
- On failure → button shows red "Failed — Try Again"
- Maximum 6 pending masjids shown, with "View All" link if more exist

24.3 Activation Logic (Backend Reference)

The existing `process_signal()` in `services.py` already handles this:

```
def process_signal(signal):
    # ...
    elif signal.sourceType == "USER":
        unique_users = _count_unique_signal_users(masjid)
        if unique_users >= MIN_COMMUNITY_SIGNALS and record.confidenceLevel < 1:
            record.confidenceLevel = 1
            record.decayDate = calculate_decay_date(1)
            record.save()
    # ...
```

Important: The current backend sets `confidenceLevel` to C1 but does not toggle `isActive` . See the Backend Suggestions section below for the recommended changes.

24.4 Suggested Backend Changes

The following backend modifications are recommended to fully support the report + verify flow. **These are suggestions only** — they have *not* been implemented.

#	CHANGE	FILE	DETAILS
1	Auto-activate on C1	<code>services.py</code>	In <code>process_signal()</code> , after setting <code>confidenceLevel = 1</code> , also set <code>masjid.isActive = True</code> and save.
2	Auto-activate on ADMIN signal	<code>services.py</code>	In the ADMIN branch, set <code>masjid.isActive = True</code> when upgrading to C2.
3	PDF upload field	<code>models.py</code>	Add <code>letterheadPDF = models.FileField(upload_to='letterheads/', blank=True, null=True)</code> to <code>MasjidAdmin</code> model.
4	PDF upload endpoint	<code>views.py</code> , <code>serializers.py</code>	Add a <code>@action(detail=True, methods=['post'])</code> on <code>MasjidAdminViewSet</code> to accept multipart PDF upload. Update serializer to include <code>letterheadPDF</code> .

5	Media file serving	<code>settings.py</code> , <code>urls.py</code>	Configure <code>MEDIA_ROOT</code> and <code>MEDIA_URL</code> . Add <code>static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)</code> in dev URLs.
6	Registration with password hashing	<code>views.py</code> , <code>models.py</code>	Currently <code>RegularUser</code> doesn't extend Django's <code>AbstractUser</code> . For JWT to work, users need to be created via Django's user model or a custom auth backend that validates against <code>RegularUser</code> . Recommended: Either (a) make <code>RegularUser</code> extend <code>AbstractUser</code> , or (b) create a custom <code>ModelBackend</code> that authenticates against <code>RegularUser</code> and stores hashed passwords.
7	Registration endpoint	<code>views.py</code>	Override <code>RegularUserViewSet.create()</code> to accept <code>password</code> , hash it with <code>make_password()</code> , and create the user. Set <code>permission_classes = [AllowAny]</code> for the <code>create</code> action only.
8	Unique signal enforcement	<code>views.py</code> or <code>services.py</code>	Prevent the same user from sending duplicate signals for the same masjid within the 30-day window. Add a unique constraint or check in <code>perform_create()</code> .